

Developing Full Stack Next.js Web Applications

Dr. Jose Annunziato

Table of Contents

Chapter 1 - Building Next.js User Interfaces with HTML.....	4
1.1 Learning Objectives.....	5
1.2 Setting Up the Development Environment.....	5
1.3 Introduction to HTML.....	9
1.4 Prototyping the React Kambaz User Interface with HTML.....	20
1.5 Committing Code to Source Control.....	33
1.6 Deploying Next.js Projects to the Web.....	36
1.7 Conclusion.....	36
Chapter 2 - Styling Web Pages with CSS.....	37
2.1 Styling React Components with CSS (Cascading Style Sheets).....	37
2.2 Decorating Documents with React Icons.....	53
2.3 Styling Webpages with the React Bootstrap CSS Library.....	54
2.4 Styling Kambaz with CSS and Bootstrap.....	66
2.6 Delivery.....	79
Chapter 3 - Creating Single Page Applications with React.....	81
3.1 Learning Objectives.....	81
3.2 Introduction to JavaScript.....	81
3.3 JavaScript Functions.....	86
3.4 JavaScript Data Structures.....	88
3.5 Dynamic Styling.....	98
3.6 Parameterizing Components.....	100
3.7 Rendering a Data Structure.....	104
3.8 Debugging.....	105
3.8 Implementing a Data Driven Kambaz Application.....	106
3.9 Deliverables.....	113
Chapter 4 - Maintaining State in React Applications.....	115
4.1 Learning Objectives.....	115
4.2 Managing State and User Input with Forms.....	115
4.3 Managing Application State with Redux.....	124
4.4 Adding State to the Kambaz User Interface.....	133
4.5 Deliverables.....	151
Chapter 5 - Implementing RESTful Web APIs with Express.js.....	152
5.1 Installing and Configuring an HTTP Web Server.....	152
5.2 Lab Exercises.....	157
5.3 Implementing the Kambaz Node.js HTTP Server.....	181
5.4 Deploying RESTful Web Service APIs to a Public Remote Server.....	204
5.5 Conclusion.....	207
5.6 Deliverables.....	207
Chapter 6 - Integrating React with MongoDB.....	208
6.1 Working with a Local MongoDB Instance.....	209
6.2 Programming with a MongoDB Database.....	211

6.3 Integrating with MongoDB Hosted in Atlas Cloud Service.....	225
6.4 Integrating the Kambaz Web Application with a Database.....	227
6.5 Deliverables.....	246
7 References.....	247

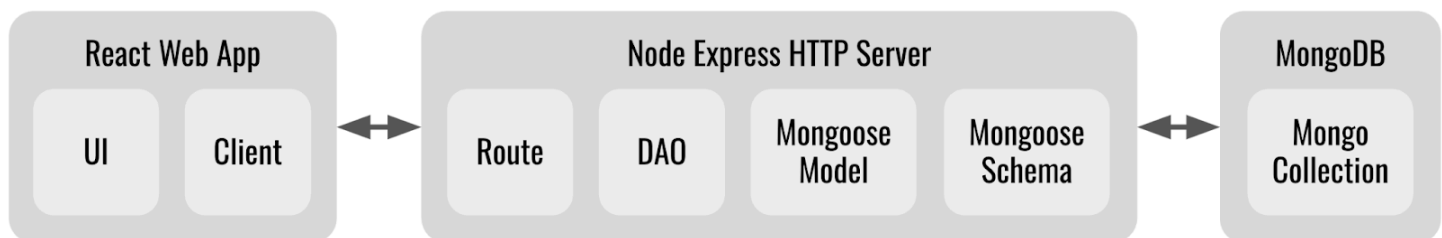
Chapter 6 - Integrating React with MongoDB

There are two main categories of databases: **relational databases** and **non-relational databases**. Relational databases such as **MySQL**, **SQL Server**, and **Postgres**, store data in **tables** containing **records** of the same **type**, e.g., a table called **courses** would contain records representing all the **courses**, and a **users** table would contain all the **users** of an application. Records are represented as **rows** in the tables where each **column** stores data for attributes specific to the type of the table, e.g., the rows in the **courses** table might have columns such as **name**, **description**, **startDate**, **endDate**, etc. Some of the columns might refer, or **relate** to other records in other tables such as the **instructor** column in the **courses** table might refer to, or relate to a particular row in the **users** table signifying that that particular user is the **instructor** of that particular course. Rows in one table relating to rows in another table is where **relational** databases get their name. The **structured query language** or **SQL**, is a computer language commonly used to interact with relational databases. The **query** in **SQL** generally means to **ask for**, or **retrieve** data that matches some criteria, often written as a **boolean expression** or **predicate**.

More recently there has been a growing interest in representing and storing data using alternative strategies which have collectively come to be referred to as **non relational databases**, or **NoSQL databases**. Non relational databases such as **MongoDB**, **Firebase**, and **Couchbase**, store their data in **collections** containing **documents** which are roughly analogous to **tables** and **records** in their relational counterparts. The biggest difference though is that the columns, or **fields** in the rows in relational databases generally can only contain **primitive data types**, e.g., simple strings, numbers, dates, and booleans, whereas the fields, or **properties** in non relational documents can be arbitrarily **complex data types**, e.g., strings, numbers, booleans, dates as well as combinations of these in complex objects containing arrays of objects of arrays, etc. The other big difference is that relational databases require the structure, or **schema** of the data to be explicitly described before storing any data, whereas non relational databases do not require predefined schemas. Instead, non relational databases delegate this responsibility to the applications using the database. The **structure**, or schema in relational databases is where **structured query language** gets its name.

In the previous chapter we learned how to create an HTTP server with Node.js and integrated it with a React Web user interface application to store the application state on the server. In this chapter we expand on this idea to store the data to **MongoDB**, a popular non relational database. The first section demonstrates how to download, install and use a local instance of the MongoDB database. The next section covers how to use the **Mongoose** library to integrate and program a MongoDB database with a Node.js server application. The final section describes how to deploy the database to **Mongo Atlas**, a remote MongoDB database hosted as a cloud service.

The following figure illustrates the overall architecture of what we'll be building in this chapter. From right to left, we'll first create a MongoDB database called **kambaz** where we'll create several **collections** such as **users**, **courses**, **modules**, **assignments**, etc. We'll use the **Mongoose** library to connect to the database programmatically from a Node server. A Mongoose schema will describe the structure of the collections in the MongoDB database and a Mongoose model will implement generic **CRUD** operations. We'll create higher level functions in **Data Access Objects (DAOs)** that operate on the database, and expose those operations through Express routes as RESTful Web APIs. A React Web app will integrate with the RESTful API through a client that will allow the user interface to interact with the database.

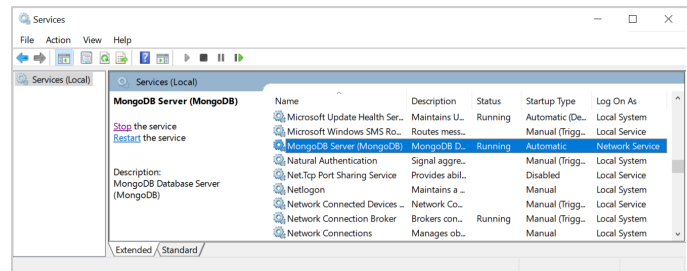


6.1 Working with a Local MongoDB Instance

MongoDB is one of an increasingly popular family of non relational databases. Data is stored in collections of documents usually formatted as JSON objects which makes it very convenient to integrate with JavaScript based frameworks such as Node.js and React. This section describes how to install, configure and get started using MongoDB.

6.1.1 Installing and Configuring MongoDB

To get started, [download MongoDB for free](#) selecting the latest version for your operating system, and click **Download**. Run the installer and, if given the choice, choose to **run the database as a service** so that you don't have to bother having to restart the database sever every time you login or restart your computer. The **MongoDB** database will automatically start whenever you start your computer. On Windows, confirm the database is running by searching for **MongoDB** in the **Services** dialog. On macOS, confirm the database is running by clicking the **MongoDB** icon in the **Systems Settings** dialog. The service dialog gives you controls to start and stop the database, but it should already be configured to start automatically when you restart your computer.



6.1.1.1 Installing MongoDB Manually (optional)

On **macOS** you can install **MongoDB** using **brew** by typing the following at the command line

```
brew install mongodb-atlas
atlas setup
```

Alternatively you can unzip the MongoDB server from the downloaded archive to a local file system and add the right commands to your operating system **PATH** environment variable. On **macOS**, unzip the file into **/usr/local** which creates a directory such as **/usr/local/mongodb-macos-x86_64-5.0.3** (your version might differ). To be able to execute the database related commands, add the path to the **.bash_profile** or **.zshrc** file located in your home directory. Add the following line in the configuration file as shown below. Your actual version might differ.

```
~/.bash_profile or ~/.zshrc
```

```
export PATH="$PATH:/usr/local/mongodb-macos-x86_64-5.0.3/bin"
```

If the **.bash_profile** or **.zshrc** file does not exist in your home directory, create it as a plain text file, but with no extensions and a period in front of it. Configure it as shown above and then **restart your computer**.

On **Windows**, unzip the file into **C:\Program Files**. To configure environment variables on **Windows** press the **Windows + R** key combination to open the **Run** prompt, type **sysdm.cpl** and press **OK**. In the **System Properties** window that appears, press the **Advanced** tab and then the **Environment Variables** button. In the **Environment Variables** configuration window select the **Path** variable and press the **Edit** button. Copy and paste the path of the **bin** directory in the **mongodb** directory you unzipped the MongoDB download, e.g., **"C:\Program Files\mongodb-macos-x86_64-5.0.3\bin"**. The actual path might differ. Press **OK** and **restart the computer**.

6.1.1.2 Starting MongoDB from the Command Line

If you installed **MongoDB** as a service, it is already running in the background and can be configured and restarted in **Windows** from the **Services** dialog or from the **System Settings** on **macOS**. Alternatively you can start the MongoDB server from the command line using the **mongod** executable in the **bin** directory where you installed **MongoDB**. First you'll need

to create a **data** folder where the server will store all its data. You can create a data folder in your home directory as shown below.

```
cd ~
mkdir data
```

When you start **MongoDB**, you'll need to tell it where the **data** folder is with the **dbpath** option. If you installed MongoDB on **Windows** in **C:\Program Files\mongodb-macos-x86_64-5.0.3**, you can start MongoDB from your home directory as shown below.

```
cd ~
C:\Program Files\mongodb-macos-aarch64--8.2.1\bin\mongod --dbpath data
```

Make sure to include the **dbpath data** option to tell **MongoDB** where to find the **data** directory.

If you installed **MongoDB** on **macOS** in **/usr/local/mongodb-macos-x86_64-6.0.5**, you can start **MongoDB** from your home directory as shown below.

```
cd ~
/usr/local/mongodb-macos-aarch64--8.2.1/bin/mongod --dbpath data
```

6.1.2 Using MongoDB Compass to Interact with MongoDB

Your installation should have installed **MongoDB Compass**, a user interface client to the MongoDB database. If not, [MongoDB Compass can be downloaded from MongoDB's download page](#). You can start **Compass** from your applications folder, or search for it in your operating system's search feature. On **macOS** bring up **Spotlight** by pressing the magnifying glass on the top right menu bar, or press the **Command** (**⌘**) and **Spacebar**. Type MongoDB **Compass** in the search bar and select the application from the result list. On **Windows** press the **Window key** to bring up the search field, type MongoDB **Compass**, and select the application from the result list. When Compass comes up, confirm that the connection string **mongodb://127.0.0.1:27017** appears in the New Connection screen, and press **Connect** to connect to MongoDB.

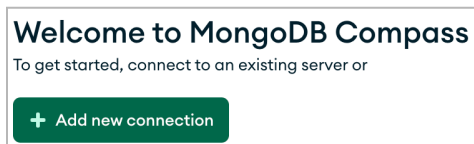


Figure 6.1.2a - Add new connection dialog box

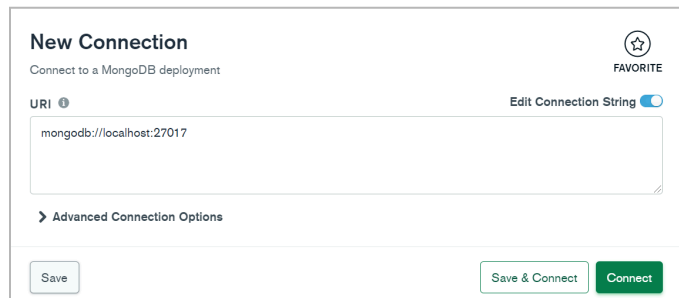
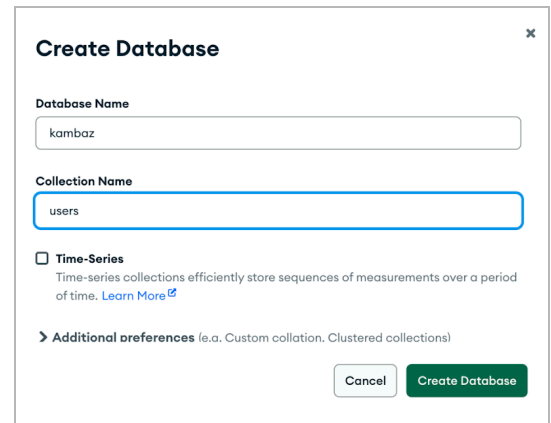
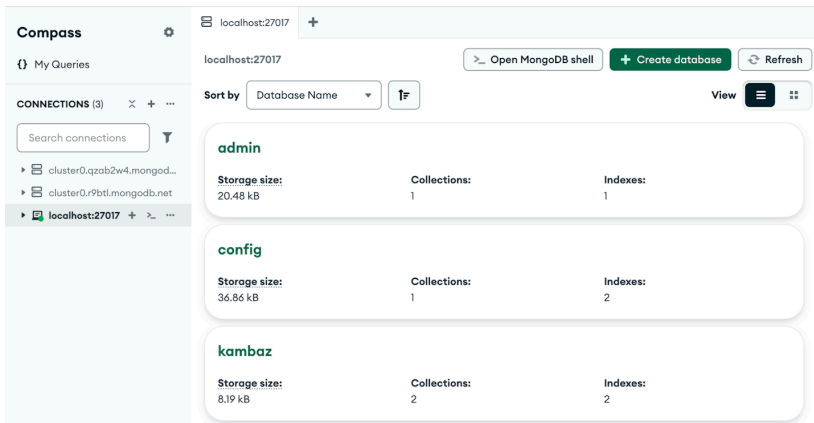


Figure 6.1.2b - Configure new connection

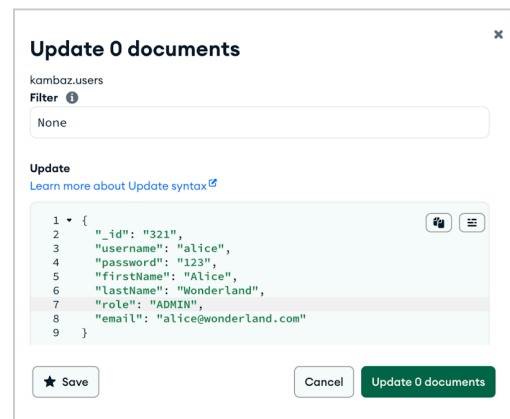
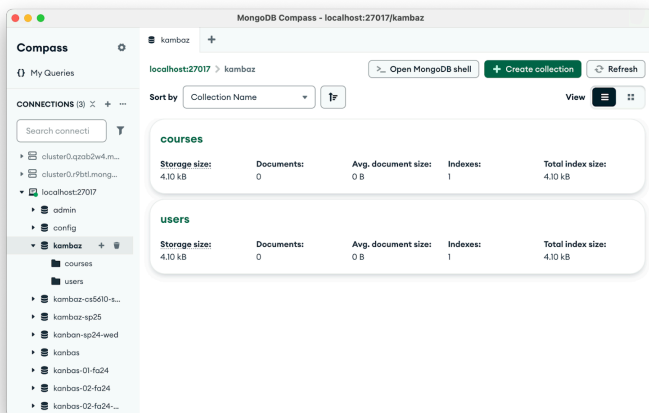
6.1.3 Creating a MongoDB Database

Once connected to a running MongoDB server, click on the connection on the left side bar and then click the **Create database** button in the tab on the right. In the **Create Database** dialog that appears, name your database **kambaz** and your first collection as **users**. Click **Create Database** to create the **kambaz** database.



6.1.4 Inserting and Retrieving Data with Compass

In MongoDB, data is organized into **collections**, which are analogous to **tables** in relational databases. Data contained in collections are referred to as **documents**, which are analogous to **records** in relational databases. To create, or **insert** documents into a collection in a **MongoDB** database using **Compass**, select the database on the left sidebar and then select the collection to insert documents into. For instance, select the **kambaz** database and then the **users** collection as shown below. On the right side, selected **ADD DATA** and then **Insert document**. In the **Insert Document** dialog that appears, insert the document as shown below. Click **Insert** to insert the document and confirm the document inserted as expected.



Instead of inserting one document at a time, entire JSON files can be imported all at once. Import the [users.json](#) file we used in earlier chapters under the **Database** directory of the React project. To import click **ADD DATA**, but now select **Import JSON or CSV file**. Navigate to the location of **users.json**, select the file and click **Import**. Confirm the users are imported. Also find the following collections and import the JSON files linked to each of the collection names. Confirm all collections are imported: [modules.json](#), [assignments.json](#), [courses.json](#).

6.2 Programming with a MongoDB Database

In the previous section we practiced interacting with the **MongoDB** database through the Compass graphical interface. This is all and good to make occasional simple manual updates and queries to confirm the data behaves as expected, but to applications need to interact with the database programmatically with libraries such as **Mongoose**. The following sections describe how to install, configure, and connect a Node.js application to a **MongoDB** database server using the **Mongoose** library. The final section discusses how to configure the application to integrate to a **MongoDB** database

hosted in the MongoDB's **Atlas** cloud service. Do all your work in a new GitHub branch called **a6** in both the React and Node.js projects.

6.2.1 Installing and Connecting to a MongoDB Database

The **Mongoose** library implements a set of APIs and abstractions for applications to interact with a MongoDB database. To use the **Mongoose** library, install it from the root of the Node.js project as shown below.

```
$ npm install mongoose
```

To connect to the database server programmatically, import the Mongoose library and then use the **connect** function as shown below. The URL in the **connect** function is called the **connection string** and is currently referring to a **MongoDB** server instance running in the **localhost** machine (the current laptop or desktop) listening at port **27017** and the **kambaz** database existing in that server. In a later section we'll revisit the connection string and configure it to connect to a database server running in a remote machine hosted by MongoDB's **Atlas** cloud service.

index.js

```
import express from "express";
import mongoose from "mongoose"; // Load the mongoose Library
...
const CONNECTION_STRING = "mongodb://127.0.0.1:27017/kambaz"
mongoose.connect(CONNECTION_STRING); // connect to the kambaz database
const app = express();
...
```

6.2.2 Configuring Connection Strings as Environment Variables

Instead of hard coding the **connection string** in the source code, it is better to configure it as an **environment variable** and then reference it from the code. This will come in handy when the server application is deployed to a remote service such as **Render** or **Heroku** and the **connection string** can be configured to reference the online remote database running on MongoDB's **Atlas** cloud service. In the **.env** file, declare the following connection string environment variable.

.env

```
SERVER_ENV=development
CLIENT_URL=http://localhost:3000
SERVER_URL=http://localhost:4000
SESSION_SECRET=super secret session phrase
DATABASE_CONNECTION_STRING=mongodb://127.0.0.1:27017/kambaz
```

Then in **index.js**, read the **connection string** as shown below.

index.js

```
import "dotenv/config";
import express from "express";
import mongoose from "mongoose";
...
const CONNECTION_STRING = process.env.DATABASE_CONNECTION_STRING || "mongodb://127.0.0.1:27017/kambaz"
mongoose.connect(CONNECTION_STRING);
const app = express();
app.use(cors({...}));
app.use(session({...}));
app.use(express.json());
...
app.listen(process.env.PORT || 4000);
```


6.2.3 Implementing Mongoose Schemas and Models

As mentioned earlier, non relational database do not require specifying the structure, or schema of the data stored in collections like relational databases do. That responsibility has been delegated to applications using non relational databases. Once a Node.js application establishes a connection to a MongoDB database, the **Mongoose** API declares datatypes **Schemas** and **Models** to interact with collections. Mongoose **Schemas** describe the structure of the data being stored in the database and it's used to validate the data being stored or modified through the Mongoose library. The **schema** shown below describes the structure for the **users** collection imported earlier. Create the schema in a **Users** directory in your Node.js projects.

kambaz/users/schema.js

```
import mongoose from "mongoose"; // Load the mongoose library
const userSchema = new mongoose.Schema({ // create the schema
  _id: String, // primary key name is _id of type String
  username: { type: String, required: true, unique: true }, // String field that is required and unique
  password: { type: String, required: true }, // String field that is required but not unique
  firstName: String, // String fields
  email: String, // with no additional
  lastName: String, // configurations
  dob: Date, // Date field with no configurations
  role: { // String field
    type: String, // allowed string values
    enum: ["STUDENT", "FACULTY", "ADMIN", "USER"], // default value if not provided
    default: "USER",
  },
  loginId: String,
  section: String,
  lastActivity: Date,
  totalActivity: String,
}, { collection: "users" }); // store data in "users" collection
export default userSchema;
```

6.2.4 Implementing Mongoose Models

Mongoose **models** implement a low level API to interact with MongoDB collections programmatically. Models provide **CRUD (Create Read Update Delete)** functions such as: **find()**, **create()**, **updateOne()**, **removeOne()**, etc. In **Users/model.js** below, create a Mongoose model from the users schema. The functions provided by Mongoose models are deliberately generic because they can interact with any collection configured in the schema. In the next section we'll create a **data access object** that implements higher level functions specific to the domain of **kambaz**.

kambaz/users/model.js

```
import mongoose from "mongoose"; // Load mongoose library
import schema from "../schema.js"; // Load users schema
const model = mongoose.model("UserModel", schema); // create mongoose model from the schema
export default model; // export so it can be used elsewhere
```

6.2.5 Retrieving Data from MongoDB with Mongoose

The Mongoose model created in the previous section provides low level functions such as **find**, **create**, **updateOne**, and **deleteOne**, that are deliberately vague since they need to be able to operate on any collection. It is good practice to wrap these low level generic functions within higher level functions that are specific to the use cases of the specific projects. For instance instead of just using the generic **find()** function, it would be preferable to use something such as **findUsers()**, **findUserById()** or **findUserByUsername()**. A previous chapter implemented a **data access object** using arrays declared in

the **Database/index.js** files. This chapter refactors the **DAOs** so they use an actual database. The following **Users/dao.js** re-implements the **CRUD** operations for the **users** collection written in terms of the low level Mongoose model operations.

kambaz/users/dao.js

```
import model from "../model.js";
import { v4 as uuidv4 } from "uuid";
export default function UsersDao(db) {
  let { users } = db;
  const createUser = (user) => {} // implemented Later
  const findAllUsers = () => model.find();
  const findUserById = (userId) => model.findById(userId);
  const findUserByUsername = (username) => model.findOne({ username: username });
  const findUserByCredentials = (username, password) => model.findOne({ username, password });
  const updateUser = (userId, user) => model.updateOne({ _id: userId }, { $set: user });
  const deleteUser = (userId) => model.deleteOne({ _id: userId });
  return { createUser, findAllUsers, findUserById, findUserByUsername, findUserByCredentials, updateUser, deleteUser };
}
```

6.2.6 Implementing APIs to Interact with MongoDB from a React Client Application

DAOs implement an interface between an application and the low level database access, providing a high level API to the rest of the application hiding the details and idiosyncrasies of using a particular database vendor. Likewise routes implement an interface between the HTTP network world and the JavaScript functional programming world by converting a stream of bits from a network connection request into a set of objects, maps, and function event handlers that participate in the client/server architecture of a multi tiered application.

The following sections demonstrate implementing the most **CRUD** (**Create Read Update** and **Delete**) database operations including:

- Retrieving all documents
- Retrieving documents by predicate
- Retrieving documents by primary Key
- Deleting a document
- Updating a document
- Creating a new documents

6.2.6.1 Refactoring Account Routes

Previous chapters implemented account routes such as **signin** and **signup** shown below. Since the **DAO** implementations used data structures imported from the local file system, the operations were **synchronous**. Now that the **DAO** is interacting with a database, the operations are **asynchronous** and must be tagged with the **async/await** keywords as shown below. Confirm **Signin**, **Signup**, and **Profile** screens work as before. Following the examples below for the **signin** and **signup** functions, add the **async** keyword to all other router functions and add the **await** keyword to all calls to **dao** functions.

kambaz/users/routes.js

```
import UsersDao from "../dao.js";
export default function UserRoutes(app, db) {
  const dao = UsersDao(db);
  ...
  const signin = async (req, res) => {
    const { username, password } = req.body;
    const currentUser = await dao.findUserByCredentials(username, password);
    if (currentUser) {
      req.session["currentUser"] = currentUser;
      res.json(currentUser);
    }
  };
}
```

```

    } else {
      res.status(401).json({ message: "Unable to login. Try again later." });
    }
  };
  const signup = async (req, res) => {
    const user = await dao.findUserByUsername(req.body.username);
    if (user) {
      res.status(400).json({ message: "Username already taken" });
      return;
    }
    const currentUser = await dao.createUser(req.body);
    req.session["currentUser"] = currentUser;
    res.json(currentUser);
  };
  ...
  app.post("/api/users/signin", signin);
  app.post("/api/users/signup", signup);
}

```

6.2.6.2 Retrieving All Documents from MongoDB with Mongoose

DAOs implement high level data operations based on lower level **Mongoose** models. The **Mongoose** model **find** function retrieves all documents from a collection. The higher level **findAllUsers** function below uses the lower level **find** function to retrieve all the users from the users collection.

kambaz/users/dao.js

```

import model from "../model.js";
...
const findAllUsers = () => model.find();

```

Routes implement **RESTful Web APIs** that user interface clients can use to interact with server functionality. The route implemented below uses the **findAllUsers** function implemented by the DAO to retrieve all the users from the database. The route responds with the collection of users retrieved from the database. Confirm the route works by navigating to <http://localhost:4000/api/users> with the browser.

kambaz/users/routes.js

```

import UsersDao from "../dao.js";
let currentUser = null;
export default function UserRoutes(app, db) {
  const dao = UsersDao(db);
  const findAllUsers = async (req, res) => {
    const users = await dao.findAllUsers();
    res.json(users);
  };
  app.get("/api/users", findAllUsers);
  ...
}

```

Meanwhile in the React user interface application, in **app/(kambaz)/account/client.ts**, implement the **findAllUsers** function shown below to send a **GET request** to the server and **await** for the server's **response** containing an array of users in the **data** property.

app/(kambaz)/account/client.ts

```

import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
export const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export const USERS_API = `${HTTP_SERVER}/api/users`;
export const findAllUsers = async () => {
  const response = await axiosWithCredentials.get(USERS_API);
  return response.data;
};

```

...

To display the array of users from the database, refactor the **PeopleTable** component to accept an optional **users** parameter, instead of retrieving the users from the local file system. Convert the PeopleTable from a page into a component as shown below. Remove any filters since data access rules is best handled at the server.

app/(kambaz)/courses/[cid]/people/Table.tsx

```
"use client"
import { useState, useEffect } from "react";
// import * as db from "../../database";
// import { useParams } from "nextjs/navigation";
export default function PeopleTable({ users = [], fetchUsers }: { users?: any[]; fetchUsers: () => void; }) {
  // const { cid } = useParams();
  // const { users, enrollments } = db;
  return (
    <div id="wd-people-table">
      <table className="table table-striped">
        ...
        <tbody>
          <tr>
            <td>
              {users
                .filter((usr) => enrollments.some((enrollment) =>
                  enrollment.user === usr.id && enrollment.course === cid))
                .map((user: any) => ( ... ))}
            </td>
          </tr>
        </tbody>
      </table>
    </div>);}
```

Create a new **Users** screen that fetches the users from the database and displays it with the **PeopleTable** component as shown below.

app/(kambaz)/account/users/page.tsx

```
"use client";
import { useState, useEffect } from "react";
import { useParams } from "next/navigation";
import PeopleTable from "../../courses/people/Table";
import * as client from "../../client";
export default function Users() {
  const [users, setUsers] = useState<any[]>([]);
  const { uid } = useParams();
  const fetchUsers = async () => {
    const users = await client.findAllUsers();
    setUsers(users);
  };
  useEffect(() => {
    fetchUsers();
  }, [uid]);
  return (
    <div>
      <h3>Users</h3>
      <PeopleTable users={users} fetchUsers={fetchUsers} />
    </div>
  );}
```

Add a **Users** link to the **Accounts Navigation** sidebar that navigates to the **Users** screen only if the logged in user has an **ADMIN** role. To test, use **Compass** to update the **role** of an existing user or create a new user with **ADMIN** role, sign in as the **ADMIN** user, and navigate to the **Users** screen. Confirm that all users are displayed.

app/(kambaz)/account/Navigation.tsx

```
"use client";
import Link from "next/link";
```

```
import { usePathname } from "next/navigation";
import { Nav, NavItem, NavLink } from "react-bootstrap";
import { useSelector } from "react-redux";

export default function AccountNavigation() {
  const { currentUser } = useSelector((state: RootState) => state.accountReducer);
  const { pathname } = usePathname();
  return (
    <Nav variant="pills">
      ...
      {currentUser && currentUser.role === "ADMIN" && (
        <NavLink as={Link} href={`/${account/users}`} active={pathname.endsWith('Users')}> Users </NavLink> )}
      ...
    </Nav>
  );
}
```

Logged in as an ADMIN, navigate to the new **Users** screen and confirm it displays all the users as shown below.

Profile

Users

Users	Name	Login ID	Section	Role	Last Activity	Total Activity
	Tony Stark	001234561S	S101	ADMIN	2020-10-01	10:21:32
	Bruce Wayne	001234561S	S101	STUDENT	2020-10-01	10:21:32
	Steve Rogers	001234561S	S101	STUDENT	2020-10-01	10:21:32

Figure 6.2.6.2 - Retrieving All Documents from MongoDB with Mongoose

6.2.6.3 Retrieving Documents by Predicate from MongoDB with Mongoose

In the **User's DAO**, implement **findUsersByRole** that filters the **users** collection by the **role** property as shown below. Mongoose model's **find** function takes as argument a JSON object used to pattern match documents in the collection. The **{role: role}** object means that documents will be filtered by their **role** property that matches the value **role**.

kambaz/users/dao.js

```
...
const findUsersByRole = (role) => model.find({ role: role }); // or just model.find({ role })
return {
  ...
  findUsersByRole
};
```

In the **User routes**, refactor the **findAllUsers** function so that it parses the **role** from the query string, and then uses the **DAO** to retrieve users with that particular role.

kambaz/users/routes.js

```
const findAllUsers = async (req, res) => {
  const { role } = req.query;
  if (role) {
    const users = await dao.findUsersByRole(role);
    res.json(users);
    return;
  }
}
```

```
const users = await dao.findAllUsers();
res.json(users);
};
```

In the React user interface application, add **findUserByRole** in the client so that it encodes the **role** in the query string of the URL as shown below.

app/(kambaz)/account/client.ts

```
export const findUsersByRole = async (role: string) => {
  const response = await
    axios.get(`${USERS_API}?role=${role}`);
  return response.data;
};
```

In the **Users** screen, add a dropdown that invokes a **filterUsers** event handler function with the selected **role**. The function updates a **role** state variable and requests from the server the list of users filtered by their role. Confirm that selecting various roles actually filters the users by their role.

app/(kambaz)/account/users/page.tsx

```
export default function Users() {
  const [users, setUsers] = useState<any[]>([]);
  const [role, setRole] = useState("");
  const filterUsersByRole = async (role: string) => {
    setRole(role);
    if (role) {
      const users = await client.findUsersByRole(role);
      setUsers(users);
    } else {
      fetchUsers();
    }
  };
  const fetchUsers = async () => { ... };
  useEffect(() => { ... }, []);
  return (
    <div>
      <h3>Users</h3>
      <select value={role} onChange={(e) => filterUsersByRole(e.target.value)}
        className="form-select float-start w-25 wd-select-role" >
        <option value="">All Roles</option>
        <option value="STUDENT">Students</option>
        <option value="TA">Assistants</option>
        <option value="FACULTY">Faculty</option>
        <option value="ADMIN">Administrators</option>
      </select>
      ...
    </div>
  );
};
```

Assistants

▼

Name	Login ID	Section	Role
 Natasha Romanoff	001234564S	S101	TA
 Aragorn Elessar	001234568S	S101	TA

Figure 6.2.6.2 - Retrieving by predicates

Now practice filtering users by their **first** or **lastName** by creating a **regular expression** used to pattern match the **firstName** or **lastName** fields of the documents in the **users** collection.

kambaz/users/dao.js

```
const findUsersByPartialName = (partialName) => {
  const regex = new RegExp(partialName, "i"); // 'i' makes it case-insensitive
  return model.find({
    $or: [{ firstName: { $regex: regex } }, { lastName: { $regex: regex } }],
  });
};
return {
  ...
  findUsersByRole,
  findUsersByPartialName
};
```

In the the **User routes**, parse a **name** parameter from the **query string** and use it to find users whose first or last names partially match the **name** parameter.

kambaz/users/routes.js

```
const findAllUsers = async (req, res) => {
  const { role, name } = req.query;
  if (role) { ... }
  if (name) {
    const users = await dao.findUsersByPartialName(name);
    res.json(users);
    return;
  }
  const users = await dao.findAllUsers();
  res.json(users);
};
```

In the React client application, implement the **findUserByPartialName** client function as shown below which encodes a **name** in the **query string** the server can use to filter users by their **first** and **lastName**.

app/(kambaz)/account/client.ts

```
export const findUsersByPartialName = async (name: string) => {
  const response = await axios.get(`${USERS_API}?name=${name}`);
  return response.data;
};
```

In the **Users** screen, create a new **name** state variable and corresponding **input** field used to invoke the **findUsersByPartialName** client function and update the **users** state variable with a subset of users that match the **name**. Confirm that typing a name in the input field actually filters the users by their first or last name. Note that the current implementation does not consider a combination of filtering by **role** and by **name**. Feel free to explore how you would go about filtering by both.

app/(kambaz)/account/users/page.tsx

```
export default function Users() {
  const [users, setUsers] = useState<any[]>([]);
  const [role, setRole] = useState("");
  const [name, setName] = useState("");
  const filterUsersByName = async (name: string) => {
    setName(name);
    if (name) {
      const users = await client.findUsersByPartialName(name);
      setUsers(users);
    } else {
      fetchUsers();
    }
  };
  ...
  return (
    <div>
```

```

    <h3>Users</h3>
    <FormControl onChange={(e) => filterUsersByName(e.target.value)} placeholder="Search people"
      className="float-start w-25 me-2 wd-filter-by-name" />
    ...
  </div>
);
}

```

All Roles
▼

Name	Login ID	Section	Role
Steve Rogers22	001234563S	S101	STUDENT
Natasha Romanoff	001234564S	S101	TA
Frodo Baggins	001234567S	S101	FACULTY

Figure 6.2.6.3 - Retrieving by regular expressions

6.2.6.4 Retrieving Documents by Primary Key from MongoDB with Mongoose

A common database operation is to retrieve documents by their primary key. The **DAO** function below retrieves a **user** document by its primary key.

kambaz/users/dao.js

```
const findUserById = (userId) => model.findById(userId);
```

Make the **findUserById** DAO function available as a RESTful Web API as shown below.

kambaz/users/routes.js

```

const findUserById = async (req, res) => {
  const user = await dao.findUserById(req.params.userId);
  res.json(user);
};
app.get("/api/users/:userId", findUserById);

```

The user interface can then interact with the server using the **findUserById** client function shown below.

app/(kambaz)/account/client.ts

```

export const findUserById = async (id: string) => {
  const response = await axios.get(`${USERS_API}/${id}`);
  return response.data;
};

```

In a new **PeopleDetails** component, use the client's **findUserById** function to retrieve the user when a faculty clicks on the user's name. Parse a **uid** path parameter and use it to retrieve the user by their ID when the component loads. If the **uid** does not exist, return **null** so that the component does not render on the screen. In **useEffect**, add **uid** as a dependency so that if the component re-renders if you click on another user while the component is still displaying.

app/(kambaz)/courses/[cid]/people/Details.tsx

```
import { useEffect, useState } from "react";
import { FaUserCircle } from "react-icons/fa";
import { IoCloseSharp } from "react-icons/io5";
import { useParams } from "next/navigation";
import Link from "next/link";
import * as client from "../../account/client";

export default function PeopleDetails({ uid, onClose }: { uid: string | null; onClose: () => void; }) {
  const [user, setUser] = useState<any>({});
  const fetchUser = async () => {
    if (!uid) return;
    const user = await client.findUserById(uid);
    setUser(user);
  };
  useEffect(() => {
    if (uid) fetchUser();
  }, [uid]);
  if (!uid) return null;
  return (
    <div className="wd-people-details position-fixed top-0 end-0 bottom-0 bg-white p-4 shadow w-25">
      <button onClick={onClose} className="btn position-fixed end-0 top-0 wd-close-details">
        <IoCloseSharp className="fs-1" /> </button>
      <div className="text-center mt-2"> <FaUserCircle className="text-secondary me-2 fs-1" /> </div><hr />
      <div className="text-danger fs-4 wd-name"> {user.firstName} {user.lastName} </div>
      <b>Roles:</b> <span className="wd-roles"> {user.role} </span> <br />
      <b>Login ID:</b> <span className="wd-login-id"> {user.loginId} </span> <br />
      <b>Section:</b> <span className="wd-section"> {user.section} </span> <br />
      <b>Total Activity:</b> <span className="wd-total-activity">{user.totalActivity}</span> </div>
    </div>
  );
}
```

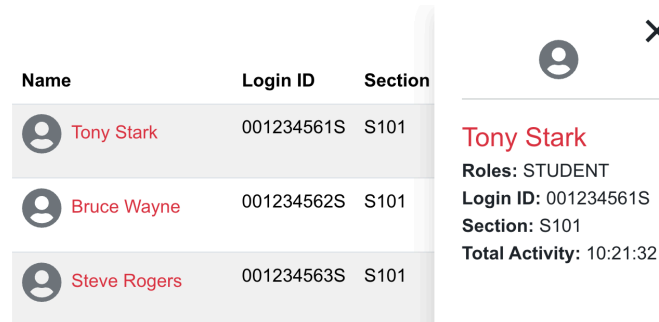


Figure 6.2.6.4 - PeopleDetails dialog

Add a **close button** rendered as an **X** at the top right that hides the component by navigating back to the **Users** screen as shown above. From **Users** page, pass **fetchUsers** to **PeopleTable** so that we can update the users if we create, update, or delete users from the **PeopleDetails** dialog. Confirm that clicking on the the name of a user displays the user's details. Also confirm that closing the **PeopleDetails** hides the component.

app/(kambaz)/courses/[cid]/people/Table.tsx

```
...
import * as client from "../../account/client";
import PeopleDetails from "../Details";
import Link from "next/link";
export default function PeopleTable({ users = [], fetchUsers }: { users?: any[]; fetchUsers: () => void; }) {
  const [showDetails, setShowDetails] = useState(false);
  const [showUserId, setShowUserId] = useState<string | null>(null);
  ...
  return (
    <div id="wd-people-table">
      {showDetails && (
        <PeopleDetails
          uid={showUserId}
        />
      )}
    </div>
  );
}
```

```

        onClose={() => {
          setShowDetails(false);
          fetchUsers();
        }}/>
      )}
      ...
    {users
      .map((user) => (
        <tr key={user._id}>
          <td className="wd-full-name text-nowrap">
            <span className="text-decoration-none"
              onClick={() => {
                setShowDetails(true);
                setShowUserId(user._id);
              }} >
              <FaUserCircle className="me-2 fs-1 text-secondary" />
              <span className="wd-first-name">{user.firstName}</span>{ " "}
              <span className="wd-last-name">{user.lastName}</span>
            </span>
          </td>
          ...
        </tr>
      )})
      ...
    </div>
  );}
};

```

6.2.6.5 Deleting a Document in MongoDB with Mongoose

To delete user documents from the **users** MongoDB collection, implement the **deleteUser** operation as shown below. The **DAO** function below removes a single **user** document from the database based on its primary key.

kambaz/users/dao.js

S

```
const deleteUser = (userId) => model.findByIdAndDelete( userId );
```

The route below makes the **deleteUser** operation available as a RESTful Web API for integration with the user interface which encodes the id of the user to remove as a path parameter.

kambaz/users/routes.js

S

```
const deleteUser = async (req, res) => {
  const status = await dao.deleteUser(req.params.userId);
  res.json(status);
};
app.delete("/api/users/:userId", deleteUser);

```

In the React Web App, implement client function that integrates with the **deleteUser** route in the server.

app/(kambaz)/account/client.ts

```
export const deleteUser = async (userId: string) => {
  const response = await axios.delete( `${USERS_API}/${userId}` );
  return response.data;
};

```

In the **PeopleDetails** component add buttons **Cancel** and **Delete** as shown below. The **Delete** button invokes a new **deleteUser** event handler function with **uid**, the ID of the user to delete. Pass a reference to **fetchUsers** as a parameter so that **PeopleDetails** can notify **PeopleTable** that a user has been remove and that the list of users must be updated.

app/(kambaz)/courses/[cid]/people/Details.tsx

```

...
export default function PeopleDetails(...) {
  const deleteUser = async (uid: string) => {
    await client.deleteUser(uid);
    onClose();
  };
  ...
  return (
    <div className="...">
      ...
      <hr />
      <button onClick={() => deleteUser(uid)} className="btn btn-danger float-end wd-delete" > Delete </button>
      <button onClick={onClose}
        className="btn btn-secondary float-end me-2 wd-cancel" > Cancel </button>
    </div>
  );
}

```

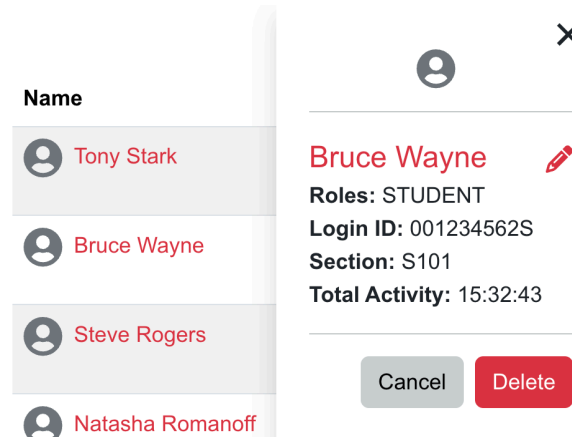


Figure 6.2.6.5 - Deleting a Document in MongoDB with Mongoose

Use the *client's* **deleteUser** to remove the user, and navigate back to hide the details component. The **Cancel** button just hides the details without removing any documents. Confirm that clicking the **Cancel** and **Delete** buttons actually work.

6.2.6.6 Updating a Document in MongoDB with Mongoose

The **Mongoose update** function updates documents in **MongoDB** databases. In the *User's DAO*, implement **updateUser** as shown below to update a single document by first identifying it by its primary key, and then updating the matching fields in the **user** parameter.

```

kambaz/users/dao.js
...
import model from "../model.js";
...
const updateUser = (userId, user) => model.updateOne({ _id: userId }, { $set: user });

```

In the *User's routes*, make the **DAO** function available as a RESTful Web API as shown below. Map a route that accepts a user's primary key as a path parameter, passes the ID and request body to the DAO function and responds with the status.

```

kambaz/users/routes.js
...
import UsersDao from "../dao.js";
export default function UserRoutes(app, db) {
  const dao = UsersDao(db);
  ...
  const updateUser = async (req, res) => {
    const { userId } = req.params;
    const userUpdates = req.body;
    await dao.updateUser(userId, userUpdates);
  };
}

```

```

    const currentUser = req.session["currentUser"];
    if (currentUser && currentUser._id === userId) {
      req.session["currentUser"] = { ...currentUser, ...userUpdates };
    }
    res.json(currentUser);
  };
  app.put("/api/users/:userId", updateUser);
  ...
}

```

In the React client application, the client function **updateUser** sends user updates to the server to be saved to the database.

app/(kambaz)/account/client.ts

```

export const updateUser = async (user: any) => {
  const response = await axiosWithCredentials.put(`${USERS_API}/${user._id}`, user);
  return response.data;
};

```

In the **PeopleDetails** component, add a **name** state variable to edit the **first** and **last** name of the user. Also add a **editing** state variable to toggle the input field that edits the **name**. With the **navigate** routing function, navigate back to **PeopleTable** once the update is done. Create a new **saveUser** function that splits the **name** state variable into the **firstName** and **lastName** and sends an updated version of the user to the server. Also update the local **user** state variable, turn off editing, and **navigate** back to the **PeopleTable**.

app/(kambaz)/courses/[cid]/people/Details.tsx

```

...
import { FaPencil } from "react-icons/fa6";
import { FaCheck, FaUserCircle } from "react-icons/fa";
export default function PeopleDetails(...) {
  ...
  const [name, setName] = useState("");
  const [editing, setEditing] = useState(false);
  const saveUser = async () => {
    const [firstName, lastName] = name.split(" ");
    const updatedUser = { ...user, firstName, lastName };
    await client.updateUser(updatedUser);
    setUser(updatedUser);
    setEditing(false);
    onClose();
  };
  ...
  return (
    ...
    <div className="text-danger fs-4">
      {!editing && (
        <FaPencil onClick={() => setEditing(true)}
          className="float-end fs-5 mt-2 wd-edit" /> )}
      {editing && (
        <FaCheck onClick={() => saveUser()}
          className="float-end fs-5 mt-2 me-2 wd-save" /> )}
      {!editing && (
        <div className="wd-name"
          onClick={() => setEditing(true)}>
          {user.firstName} {user.lastName}</div>)}
      {user && editing && (
        <FormControl className="w-50 wd-edit-name"
          defaultValue={`${user.firstName} ${user.lastName}`}
          onChange={(e) => setName(e.target.value)}
          onKeyDown={(e) => {
            if (e.key === "Enter") { saveUser(); }}} />)}
    </div>
    ...
  );
}

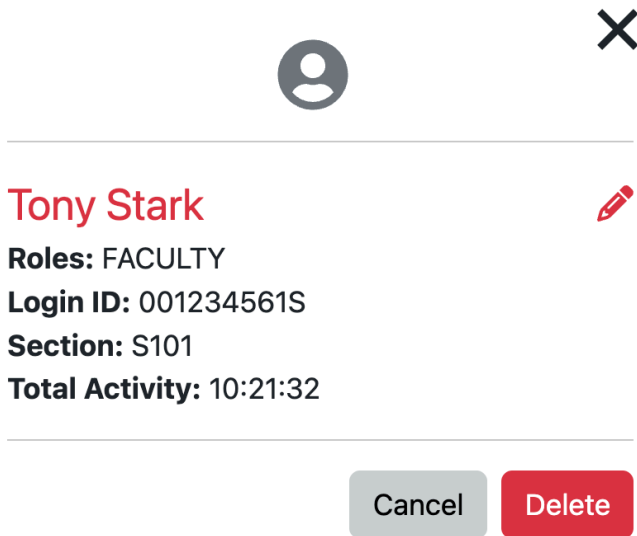
```

// to edit the user's first and last name
// whether we are editing or not
// to save updates to user's name
// split the name into an array and get first
// and last and create new version of user overriding
// first and last. Send update to server
// update local copy of the user
// we're done editing
// close the dialog

// if not editing show pencil icon
// clicking pencil turns on editing and hides pencil

// if editing show check mark. Clicking check turns
// off editing, saves and hides check
// if not editing show first and last name
// clicking on name turns on editing

// if editing show input field to edit name
// name is initially concatenation of first and last
// update name as we type
// save the user if Enter key is pressed



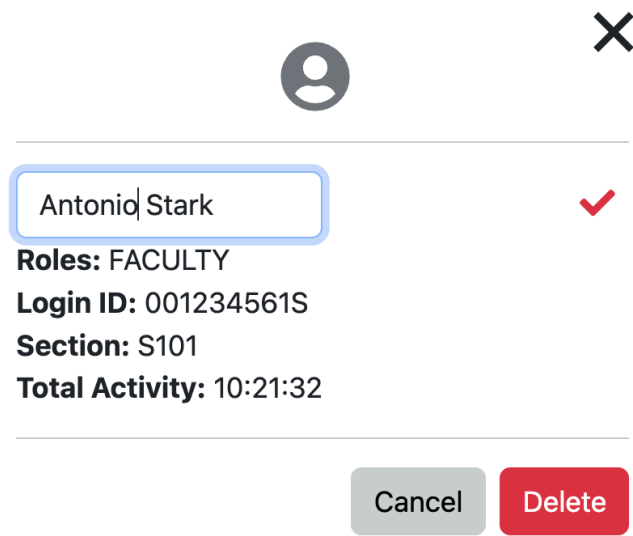



Tony Stark


Roles: FACULTY
Login ID: 001234561S
Section: S101
Total Activity: 10:21:32

Cancel Delete

Figure 6.2.6.6a - Updating a Document in MongoDB






Antonid Stark
 

Roles: FACULTY
Login ID: 001234561S
Section: S101
Total Activity: 10:21:32

Cancel Delete

Figure 6.2.6.6b - Updating a Document in MongoDB

Add **pencil** and **check** icons to turn **editing** on and off. Hide each icon based on the **editing** boolean state variable. Clicking the name of the user also turns **editing** on. If **editing** is on, hide the user's name and instead display an input field that shows the current user's **firstName** and **lastName** and edits the **name** state variable. Pressing the **Enter** key saves the updated user's details. Confirm users can be edited. On your own, add the ability to edit a user's email and role. Use an input field of type **email** to edit the email. To edit the user's role, use a dropdown similar to the one used to filter users by their role.

6.2.6.7 Creating New Documents in MongoDB with Mongoose

In the **User's DAO**, implement the **createUser** function as shown below to insert a new **user** object into the **users** collection.

kambaz/users/dao.js

5

```
const createUser = (user) => {
  const newUser = { ...user, _id: uuidv4() };
  return model.create(newUser);
} // insert new user into the database
```

Make sure the incoming **user** object does not have an **_id** property since it can interfere with the database insert operation. In the **User's routes**, make the DAO operation available as a RESTful Web API for the user interface to interact with. The new user is posted to the route in the request's body. The DAO **createUser** function inserts the new user into the database and returns the newly inserted **user** which is sent back to the user interface in the response.

kambaz/users/routes.js

5

```
import UsersDao from "../dao.js";
export default function UserRoutes(app, db) {
  const dao = UsersDao(db);
  ...
  const createUser = async (req, res) => {
    const user = await dao.createUser(req.body);
    res.json(user);
  };
  app.post("/api/users", createUser);
  ...
}
```

In the React client application, implement a **createUser** client function to interact with the route created above. Post the new user object to the server as shown below.

app/(kambaz)/account/client.ts

```
export const createUser = async (user: any) => {
  const response = await axios.post(`${USERS_API}`, user);
  return response.data;
};
```

In the **Users** screen, implement a new **createUser** event handler that sends a new user object to be inserted in the database. Use default values for the fields as shown and confirm that clicking the new **+ People** button actually creates the new user. Optionally implement editing those fields in the **PeopleDetails** component.

app/(kambaz)/account/users/page.tsx

```
export default function Users() {
  const { uid } = useParams();
  const [users, setUsers] = useState<any[]>([]);
  const [role, setRole] = useState("");
  const [name, setName] = useState("");
  const createUser = async () => {
    const user = await client.createUser({
      firstName: "New",
      lastName: `User${users.length + 1}`,
      username: `newuser${Date.now()}`,
      password: "password123",
      email: `email${users.length + 1}@neu.edu`,
      section: "S101",
      role: "STUDENT",
    });
    setUsers([...users, user]);
  };
  return (
    <div>
      <button onClick={createUser} className="float-end btn btn-danger wd-add-people">
        <FaPlus className="me-2" />
        Users
      </button>
      ...
    </div>
  );
};
```

6.3 Integrating with MongoDB Hosted in Atlas Cloud Service

When you run your server on your development environment, it should be connecting to a MongoDB instance running on the same local development computer. When you deploy the server on a remote server such as **Render**, **Heroku** or **AWS**, the server needs to connect to a database that is also hosted on a public site. **MongoDB Atlas Cloud Service** provides a hosted database service where a MongoDB instances run on public servers, and they provide a connection string to integrate our Node.js application. This section describes setting up and deploying the database online and then integrating with it from our Node.js server running on Heroku.

6.3.1 Setting up MongoDB Atlas

To get started, head over to <https://www.mongodb.com/> and click on **Sign in** at the top right corner. Login with your **Google** account or click on **Sign Up** to create an account with an email and password. If you get a validation email, confirm it and login. Answer any general questions if asked during the sign up process. In the **Deploy your cluster** screen choose a **Free** plan for now which should be enough for this course. Name your cluster **Kambaz**. In the **Provider** section, choose any of the cloud providers and in the **Region** section choose a region close to your geographic area, for instance **AWS** and

North Virginia, and then click **Create Deployment**. In the **Connect to Kabas** screen, in the **Create a database user** section create credentials to login to your database. In the **Username** and **Password** fields, type credentials you'll remember later since these are the credentials **mongoose** will use to login to the database from your Node.js server application when running on Render or Heroku. If you forget these credentials you'll need to create new ones later. I went with **giuseppi** and **supersecretpassword** :) and clicked **Create Database User**.

Create a database user using a username and password. Users will be given the *read and write to any database* privilege by default. You can update these permissions and/or create additional users later. Ensure these credentials are different to your MongoDB Cloud username and password. You can manage existing users via the [Database Access Page](#).

Username
giuseppi

Password

Autogenerate Secure Password Copy

Create User

Username	Authentication Type
giuseppi	Password

EDIT REMOVE

Figure 6.3 - Integrating with MongoDB Hosted in Atlas Cloud Service

Where would you like to connect from?
Enable access for any network(s) that need to read and write data to your cluster.

My Local Environment
Use this to add network IP addresses to the IP Access List. This can be modified at any time.

Cloud Environment
Use this to configure network access between Atlas and your cloud or on-premise environment. Specifically, set up IP Access Lists, Network Peering, and Private Endpoints.

Add entries to your IP Access List

Only on IP address you add to your Access List will be able to connect to your project's clusters. You can manage existing IP entries via the [Network Access Page](#).

IP Address	Description
Enter IP Address	Enter description

Add Entry

IP Access List	Description
45.90.144.70/32	My IP Address

EDIT REMOVE

Finish and Close

Figure 6.3.1 - Setting up MongoDB Atlas

6.3.1.1 Connecting to a Remote Database from Compass

Then click the **Choose a connection method** button, and in the **Access your data through tools**, select **Compass**. In the **Connecting with MongoDB Compass** screen, in the **Copy the connection string, then open MongoDB Compass** section, copy the connection string which should look something like so

```
mongodb+srv://giuseppi:supersecretpassword@kambaz.jxui0bc.mongodb.net/
```

Click **Done**. From **Compass** select **Connect, New Window**, paste the connection string in the **URI** field, and then click connect. Following the same steps used earlier, import the **courses**, **modules**, and **users** into the remote database.

6.3.1.2 Connecting from Node.js

In the **Atlas** main window click on **Network Access** and in the **Network Access** screen **IP Access List** tab, select **+ ADD IP ADDRESS**. In the **Add IP Access List Entry** dialog click on **ALLOW ACCESS FROM ANYWHERE**. This will add **0.0.0.0/0** to the **Access List Entry**, allowing any computer to connect. Click **Confirm** and verify the new entry appears in the **Network Access** screen. Click **Database** on the left and in the **Clusters** screen, click **Connect**. In the **Connect to Kambaz** dialog, in the **Connect to your application**, select **Drivers and version** section, confirm the **Driver** is set to **Node.js** and **Version** is set to **5.5 or later**. In the **Add your connection string into your application code** section, copy the the URL. It should look similar to the following.

Node MongoDB Database Connection String

```
mongodb+srv://giuseppi:<password>@kambaz.jxui0bc.mongodb.net/kambaz?retryWrites=true&w=majority&appName=Kambaz
```

Commit and push your code to a branch called **a6** and deploy the server application to a new remote service running on **Render**, **Heroku**, or **AWS**. Make sure not to deploy to the server for prior chapters since TAs might still be grading it. In the new remote server, configure an environment variable called **DATABASE_CONNECTION_STRING** with the URL value above. For instance, in the **Render** dashboard click on **Environment**, type **DATABASE_CONNECTION_STRING** in the **Key** field and the URL in the **Value** field. Replace the **<password>** with the actual password created in an earlier step. Commit and deploy

the React application to a new **a6** branch. Configure the **NEXT_PUBLIC_HTTP_SERVER** to point to the new remote server. For the environment variables to take effect, you might need to redeploy and/or restart the remote Node server on Render as well as the remote React application server on Vercel. **NOTE:** the **DATABASE_CONNECTION_STRING** environment variable must include the name of the database at the end of the path, e.g., between the last slash (/) and the question mark (?). The above example highlights the name of the database **kambaz** in red.

Add IP Access List Entry

Atlas only allows client connections to a cluster from entries in the project's IP Access List. Each entry should either be a single IP address or a CIDR-notated range of addresses. [Learn more.](#)

ALLOW ACCESS FROM ANYWHERE

Access List Entry:0.0.0.0/0

Comment:Optional comment describing this entry

☐ This entry is temporary and will be deleted in 6 hours

CancelConfirm

Figure 6.3.1.2a - Configuring IP Access

MANAGE

Environment

Shell

Scaling

Previews

Disks

Jobs

Environment

+ Create environment group

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

Key	Value
MONGO_CONNECTION_STRING	mongodb+srv://giuseppi:supersecretpassword@kambaz.jxui0bc.mongodb.net/ kambaz ?retryWrites=true&w=majority&appName=Kambaz

Figure 6.3.1.2b - Configuring connection string environment variable in Render.com

6.3.2 Configuring Session in Remote Servers

The local Node server was configured to support multiple sessions. Similarly, the remote server also needs to be configured to support sessions. In **Render.com**, navigate to the **Environment** section of the application dashboard. Click **Add Environment Variable**, type the name of the variables in the **Key** column and the variable's value in the **Value** column. Repeat for each of the environment variables as shown here on the right. When environment variables change, the server must be restarted by clicking **Manual Deploy** and then **Deploy latest commit**. Below is an example of the environment variables used to configure a remote server running on **Render.com**. Use the same **Environment Variable** keys shown on the left column below. Don't use the values shown on the right column below. Instead use the values for your Mongo Database, your Vercel remote server, and your remote Node server. Note: **SERVER_URL** should not start with **https://**, remove it if present.

Environment Variable	Value
DATABASE_CONNECTION_STRING	mongodb+srv://giuseppi:supersecretpassword@kambaz.jxui0bc.mongodb.net/ kambaz ?retryWrites=true&w=majority&appName=Kambaz
CLIENT_URL	https://a6--kambaz-next-js-sp25.Vercel.app
SERVER_URL	kambaz-node-server-app-sp25.onrender.com
SERVER_ENV	production
SESSION_SECRET	what ever

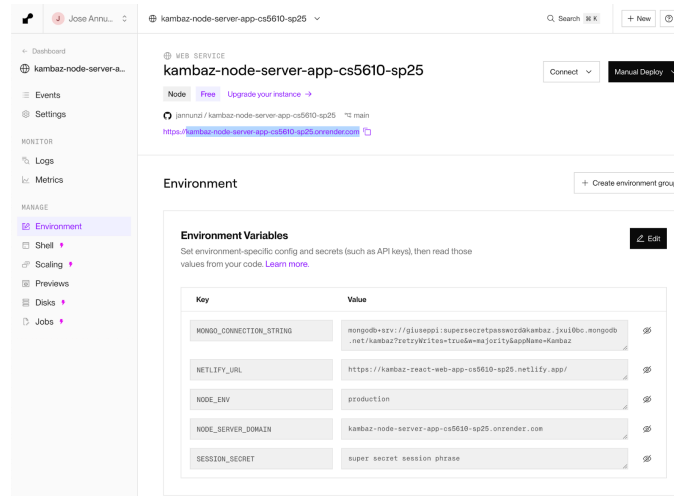


Figure - 6.3.2 Configuring Session in Remote Servers

6.4 Integrating the Kambaz Web Application with a Database

The current **Kambaz** implementation renders various courses, modules and assignments using **JSON** files. The files are wrapped into a Database data structure that first lived in the React application and then in the Node server. It is time for the data to live where it belongs. This section demonstrates migrating the **courses** and **modules** into corresponding collections in the **kambaz** MongoDB database. Create the **Mongoose** schemas, models and DAOs, and refactor the **RESTful** Web APIs to **CRUD** courses and modules in the database. Confirm that all **courses** and **modules CRUD** functionality works as expected. Optionally also migrate the assignments into a collection and confirm that all assignment **CRUD** operations work.

6.4.1 Storing Courses in a Database

The current server application implements RESTful Web APIs to access courses stored in the **courses.js** file. This section replaces the source of the courses from a **MongoDB courses** collection. The basic operations on any data source are create, read, update and delete, colloquially referred to as **CRUD** operations. The following sections demonstrate how to implement several **CRUD** operations for courses.

6.4.1.1 Retrieving Courses from a Database

To access the **courses** collection created earlier, create a **mongoose schema** file as shown below. The schema file describes the **constraints** of the **documents** stored in a **collection**, such as the names of the **properties**, their data types, and the name of the collection where the documents will be stored.

kambaz/courses/schema.js

5

```
import mongoose from "mongoose";
const courseSchema = new mongoose.Schema({
  _id: String,
  name: String,
  number: String,
  credits: Number,
  description: String,
}, {
  collection: "courses"
});
export default courseSchema;
```

Using the **mongoose schema** file, create a **mongoose model** file as shown below. Mongoose models provide functions to interact with the collection such as **find()**, **create()**, **updateOne()**, and **deleteOne()**. The **CourseModel** in the mongoose model declares a unique name that can be used as a reference from other mongoose schemas.

kambaz/courses/model.js

5

```
import mongoose from "mongoose";
import schema from "../schema.js";
const model = mongoose.model("CourseModel", schema);
export default model;
```

Using the **mongoose model**, refactor the courses **DAO** file to interact with the **courses** collection. Start by refactoring the **findAllCourses()** and **findCoursesForEnrollments** functions to retrieve courses from the database instead of the **Database** file as shown below. The **model.find()** function returns an array containing all the courses documents in the **courses** collection. We'll deal with moving enrollments to the database later in the chapter.

kambaz/courses/dao.js

5

```
import model from "../model.js";

function findAllCourses() {
  // return Database.courses;
  return model.find();
}

async function findCoursesForEnrolledUser(userId) {
  const { courses, enrollments } = db;
  const courses = await model.find();
  const enrolledCourses = courses.filter((course) =>
    enrollments.some(
      (enrollment) =>
        enrollment.user === userId && enrollment.course === course._id
    )
  );
  return enrolledCourses;
}
...
```

The functions in **mongoose models** all return **promises** allowing **asynchronous** communication with the MongoDB server. In the Courses **routes** file, redeclare all router functions as asynchronous by adding the **async** keyword in front of the router functions as shown below. Also add the **await** keyword in front of all asynchronous calls of the **DAO** functions as shown below.

kambaz/courses/routes.js

5

```
export default function CourseRoutes(app, db) {
  ...
  const findAllCourses = async (req, res) => {
    const courses = await dao.findAllCourses();
    res.send(courses);
  }
  const findCoursesForEnrolledUser = async (req, res) => {
    let { userId } = req.params;
    if (userId === "current") {
      const currentUser = req.session["currentUser"];
      if (!currentUser) {
        res.sendStatus(401);
        return;
      }
      userId = currentUser._id;
    }
    const courses = await dao.findCoursesForEnrolledUser(userId);
    res.json(courses);
  };
};
```

```
} ...
```

6.4.1.2 Inserting Courses into a Database

Refactor the **DAO's** **createCourse()** function to insert new courses into the database with the **mongoose model** as shown below.

kambaz/courses/dao.js

5

```
import model from "../model.js";
...
function createCourse(course) {
  const newCourse = { ...course, _id: uuidv4() };
  return model.create(newCourse);
  // const newCourse = { ...course, _id: uuidv4() };
  // Database.courses = [...Database.courses, newCourse];
  // return newCourse;
}
...
```

Refactor the Courses routes file by adding keywords **async** and **await** before the route and DAO functions as shown below. Create a new course from the **Dashboard** and confirm the new course appears in the database.

kambaz/courses/routes.js

5

```
import CoursesDao from "../dao.js";
export default function CourseRoutes(app, db) {
  const dao = CoursesDao(db);
  ...
  const createCourse = async (req, res) => {
    const newCourse = await dao.createCourse(req.body);
    const currentUser = req.session["currentUser"];
    enrollmentsDao.enrollUserInCourse(currentUser._id, newCourse._id);
    res.json(newCourse);
  }
  app.post("/api/courses", createCourse);
  ...
}
```

6.4.1.3 Deleting Courses from the Database

Refactor the **deleteCourse()** **DAO** function to delete courses from the database by using the courses **model** as shown below. We'll deal with moving enrollments to the database later in the chapter.

kambaz/courses/dao.js

5

```
function deleteCourse(courseId) {
  const { courses, enrollments } = db;
  db.enrollments = enrollments.filter( (enrollment) => enrollment.course !== courseId );
  // db.courses = courses.filter(({course}) => course._id !== courseId);
  return model.deleteOne({ _id: courseId });
}
```

In the Courses routes file, refactor the route that deletes courses by adding keywords **async** and **await** in front of the route and **DAO** functions as shown below. From the **Dashboard**, delete a course and confirm it no longer appears in the database.

kambaz/courses/routes.js

5

```
import CoursesDao from "../dao.js";
export default function CourseRoutes(app, db) {
```

```

const dao = CoursesDao(db);
...
const deleteCourse = async (req, res) => {
  const { courseId } = req.params;
  const status = await dao.deleteCourse(courseId);
  res.send(status);
}
app.delete("/api/courses/:courseId", deleteCourse);
...
}

```

6.4.1.4 Updating Courses in the Database

In the **Courses DAO**, refactor the **updateCourses** function to update courses in the database with the **model.updateOne()** function as shown below.

kambaz/courses/dao.js

5

```

import model from "../model.js";
function updateCourse(courseId, courseUpdates) {
  return model.updateOne({ _id: courseId }, { $set: courseUpdates });
  // const { courses } = Database;
  // const course = courses.find((course) => course._id === courseId);
  // Object.assign(course, courseUpdates);
  // return course;
}

```

In the **Courses routes**, refactor the routing function by adding **async** and **await** keywords before the routing and DAO function calls as shown below. From the **Dashboard**, edit a course and confirm it updates in the database.

kambaz/courses/routes.js

5

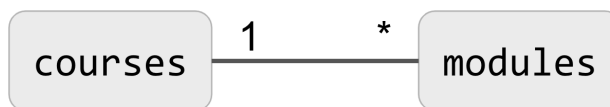
```

import CoursesDao from "../dao.js";
export default function CourseRoutes(app, db) {
  const dao = CoursesDao(db);
  ...
  const updateCourse = async (req, res) => {
    const { courseId } = req.params;
    const courseUpdates = req.body;
    const status = await dao.updateCourse(courseId, courseUpdates);
    res.send(status);
  }
  app.put("/api/courses/:courseId", updateCourse);
  ...
}

```

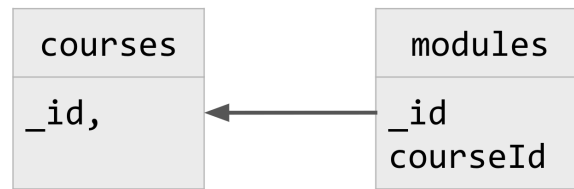
6.4.2 Persisting Modules in a Database as One to Many Relations with Courses

In **Kambaz**, each course contains several modules, establishing a **one to many relationship**. Currently the relationship is implemented by each module having a field that refers to the course they belong to. This section demonstrates how to use Mongoose to implement **one to many relationships**. In UML, one to many relationships can be illustrated as shown below.

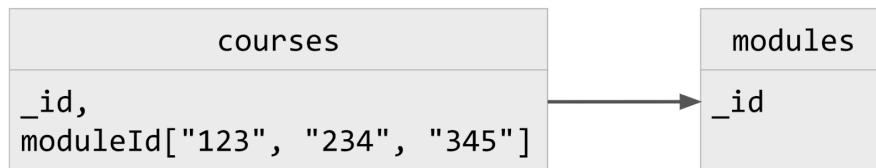


The **courses** collection is said to be on the one side of a one to many relation and the **modules** collection is said to be on the many side of a one to many relationship. The relationship describes that each course is related to many modules. It is often useful to think of the relationship as a **parent child relationship** describing it as **courses are the parents of many modules**. Another way to think of the relationship is as an ownership relationship as in **courses have many modules**. The

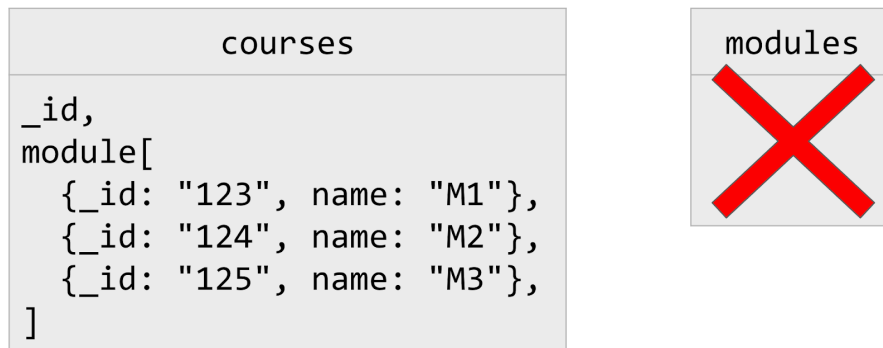
easiest way to implement one to many relationships in both relational and non-relational databases, is to use foreign keys referencing related records. In the diagram below, module documents contain field **courseId** whose value is the **_id** of some course document the module belongs to.



In non-relational databases there are two additional alternatives to implement one to many relationships. One way is to include an array of foreign keys in the parent document that reference all the child documents. In the diagram below documents in the **courses** collection contain a **moduleId** array that contains the values of primary keys of child module documents. The keys in the **moduleId** array can be used to retrieve the actual documents from the **modules** collection.



Another alternative implementation is to do away entirely with the collection on the many side (**modules**) and embed the documents in the collection on the one side (**courses**). In the diagram below the **modules** collection is removed and the documents are instead embedded in the corresponding parent course document in a **module** array.



6.4.2.1 Declaring One to Many Relationships

Since modules are not expected to be accessible outside their course, we'll embed the module documents in a new **modules** property in the **course** schema. To demonstrate, create the **schema** file below that describes the data structure of **module** documents.

kambaz/modules/schema.js

5

```
import mongoose from "mongoose";
const schema = new mongoose.Schema(
  {
    _id: String,
    name: String,
    description: String
  }
);
export default schema;
```

In the **courses** schema, add a new **modules** field defined as an array of **moduleSchema** as shown below.

kambaz/courses/schema.js

5

```
import mongoose from "mongoose";
import moduleSchema from "../modules/schema.js";
const courseSchema = new mongoose.Schema({
  _id: String,
  name: String,
  number: String,
  credits: Number,
  description: String,
  modules: [moduleSchema]
}, { collection: "courses" });
export default courseSchema;
```

Note that lessons also have a one to many relationship with the modules they belong to. Although we could create a dedicated **lessons** schema file and then add it to the **modules** schema, we can alternatively declare the **lessons** schema inline within the **modules** schema as shown below. Creating a separate dedicated lesson schema would, in general, be a better practice, but since it's a trivial schema we can get away with declaring the lesson schema right inside the parent module schema.

kambaz/modules/schema.js

5

```
import mongoose from "mongoose";
const schema = new mongoose.Schema(
  {
    _id: String,
    name: String,
    description: String,
    lessons: [{ _id: String, name: String, description: String }],
  }
);
export default schema;
```

The new schema is not compatible with the courses we imported earlier with compass. Delete all the documents from the **courses** collection, and import a [new version that embeds the lessons into their modules and the modules into their courses](#).

The **findAllCourses** and **findCoursesForEnrolledUser** in the **courses DAO** are only used in the **Dashboard** page which only really needs the course's **name** and **description**. It would be unnecessarily expensive to include the **modules** and **lessons** from the server if they are not needed in the user interface. Refactor the **findAllCourses** and **findCoursesForEnrolledUser** functions so that they only include the **name** and **description** properties in the response.

kambaz/courses/dao.js

5

```
function findAllCourses() {
  return model.find({}, { name: 1, description: 1 });
}
async function findCoursesForEnrolledUser(userId) {
  const { enrollments } = db;
  const courses = await model.find({}, { name: 1, description: 1 });
  const enrolledCourses = courses.filter((course) =>
    enrollments.some(
      (enrollment) =>
        enrollment.user === userId && enrollment.course === course._id
    )
  );
  return enrolledCourses;
}
```

In the **Dashboard**, take a look at the response from the server and confirm that the courses only contain the **name** and **description** properties.

6.4.2.2 Retrieving Modules for a Course

Refactor the **Modules DAO** so that it uses the **mongoose model** to retrieve modules for a course from the database as shown below.

kambaz/modules/dao.js

5

```
import { v4 as uuidv4 } from "uuid";
import model from "../courses/model.js";
export default function ModulesDao(db) {
  async function findModulesForCourse(courseId) {
    const course = await model.findById(courseId);
    return course.modules;
    // const { modules } = Database;
    // return modules.filter((module) => module.course === courseId);
  }
  ...
}
```

In the **Modules routes**, refactor the routing function by adding **async** and **await** keywords before the routing and DAO function calls as shown below.

kambaz/modules/routes.js

5

```
import ModulesDao from "../modules/dao.js";
export default function ModulesRoutes(app, db) {
  const dao = ModulesDao(db);
  const findModulesForCourse = async (req, res) => {
    const { courseId } = req.params;
    const modules = await dao.findModulesForCourse(courseId);
    res.json(modules);
  }
  app.get("/api/courses/:courseId/modules", findModulesForCourse);
  ...
}
```

6.4.2.3 Creating Modules for a Course

Refactor the **Modules DAO** so that it uses the **mongoose model** to insert a new module into the database as shown below.

kambaz/modules/dao.js

5

```
import { v4 as uuidv4 } from "uuid";
import model from "../courses/model.js";
export default function ModulesDao(db) {
  async function createModule(courseId, module) {
    const newModule = { ...module, _id: uuidv4() };
    const status = await model.updateOne(
      { _id: courseId },
      { $push: { modules: newModule } }
    );
    return newModule;
    // db.modules = [...db.modules, newModule];
    // return newModule;
  }
}
```

In the **Modules routes**, refactor the routing function by adding **async** and **await** keywords before the routing and DAO function calls as shown below.

```
import ModulesDao from "../modules/dao.js";
export default function ModulesRoutes(app, db) {
  const dao = ModulesDao(db);
  const createModuleForCourse = async (req, res) => {
    const { courseId } = req.params;
    const module = {
      ...req.body,
      // course: courseId,
    };
    const newModule = await dao.createModule(courseId, module);
    res.send(newModule);
  };
}
```

6.4.2.4 Deleting Modules

Refactor the **Modules DAO** so that it uses the **mongoose model** to delete modules from the database as shown below.

```
async function deleteModule(courseId, moduleId) {
  const status = await model.updateOne(
    { _id: courseId },
    { $pull: { modules: { _id: moduleId } } }
  );
  return status;
  // const { modules } = db;
  // Database.modules = modules.filter((module) => module._id !== moduleId);
}
```

In the **Modules routes**, refactor the routing function by adding **async** and **await** keywords before the routing and DAO function calls as shown below.

```
import * as modulesDao from "../dao.js";
export default function ModuleRoutes(app, db) {
  ...
  const deleteModule = async (req, res) => {
    const { courseId, moduleId } = req.params;
    const status = await modulesDao.deleteModule(courseId, moduleId);
    res.send(status);
  }
  app.delete("/api/courses/:courseId/modules/:moduleId", deleteModule);
  ...
}
```

In the **client**, make sure to encode the course's ID in the URL since the server will need the ID of the course.

```
export const deleteModule = async (courseId: string, moduleId: string) => {
  const response = await axios.delete(
    `${COURSES_API}/${courseId}/modules/${moduleId}`
  );
  return response.data;
};
```

In the **Modules** page pass the course's ID to the client in the **onRemoveModule** event handler.


```
const onRemoveModule = async (moduleId: string) => {
  await client.deleteModule(cid, moduleId);
  dispatch(setModules(modules.filter((m: any) => m._id !== moduleId)));
};
```

6.4.2.5 Updating Modules

Refactor the **Modules DAO** so that it uses the **mongoose model** to update modules in the database as shown below.

kambaz/modules/dao.js

S

```
async function updateModule(courseId, moduleId, moduleUpdates) {
  // const { modules } = db;
  // const module = modules.find((module) => module._id === moduleId);
  const course = await model.findById(courseId);
  const module = course.modules.id(moduleId);
  Object.assign(module, moduleUpdates);
  await course.save();
  return module;
}
```

In the **Modules routes**, refactor the routing function by adding **async** and **await** keywords before the routing and DAO function calls as shown below.

kambaz/modules/routes.js

S

```
import ModulesDao from "../dao.js";
export default function ModuleRoutes(app, app) {
  ...
  const updateModule = async (req, res) => {
    const { courseId, moduleId } = req.params;
    const moduleUpdates = req.body;
    const status = await modulesDao.updateModule(courseId, moduleId, moduleUpdates);
    res.send(status);
  }
  app.put("/api/courses/:courseId/modules/:moduleId", updateModule);
  ...
}
```

In the **client**, pass the course ID

app/(kambaz)/courses/client.ts

S

```
export const updateModule = async (courseId: string, module: any) => {
  const { data } = await axios.put(
    `${COURSES_API}/${courseId}/modules/${module._id}`,
    module
  );
  return data;
};
```

In the **Module** page, pass the course's ID

app/(kambaz)/courses/modules/page.tsx

S

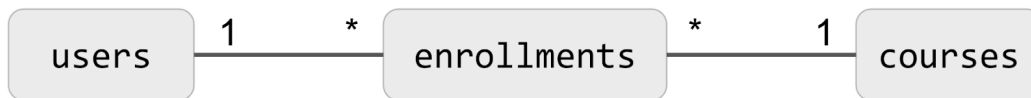
```
const onUpdateModule = async (module: any) => {
  await client.updateModule(cid, module);
  const newModules = modules.map((m: any) =>
    m._id === module._id ? module : m
  );
  dispatch(setModules(newModules));
};
```

6.4.3 Persisting Enrollments in a Database as Many to Many Relations

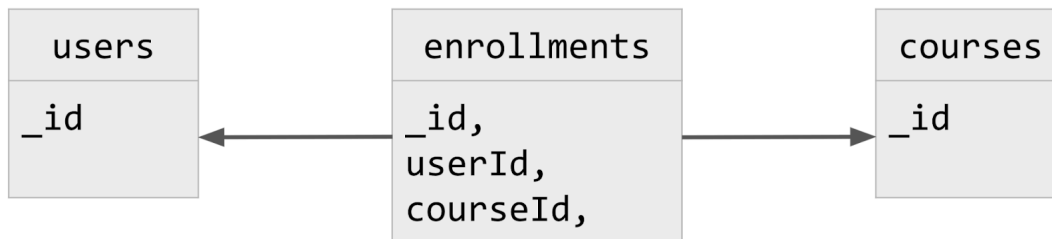
In Kambaz, a user can be enrolled in several courses, and a course can have many enrollments. An enrollment establishes a relationship between a user and a course. Since there can be many enrollments where many users can be enrolled (or associated) in many courses, the enrollments relation is referred to as a **many to many relation**. Many to many relationships can be represented in UML as shown in the following diagram capturing the fact that many users are related to many courses.



Implementing the diagram above in relational databases is challenging since it implies that each record in the **users** collection contains several references to records in the **courses** collection and vice versa. It is often easier to understand the implementation by using an intermediate collection that captures the relationship between each record in the original collections. For instance the UML diagram below uses a new **enrollments** collection that refactors the **many to many** relationship as two **one to many** relationships. The new collection is often referred to as a **mapping table** or **collection**.



This new UML representation is easier to implement as the original collections **users** and **courses** no longer need to know anything about each other and instead a new **enrollments** collection can establish the relationship by capturing references to the records that related, e.g., a record entry in the **enrollments** collection captures what users are enrolled in what courses. Practially the **enrollments** collection declares fields **userId** and **courseId** to record references of the documents that are related to each other, as shown below.



The following sections describes how to implement **many to many** relationships using **mongoose**.

6.4.3.1 Declaring Enrollments as a Many to Many Relationship

The **Courses model** below, implemented earlier, declares **CourseModel** as the name of the model for **course** documents stored in the **courses** collection. This name can be used to establish relationships between **models** and **collections**.

kambaz/courses/model.js

5

```
import mongoose from "mongoose";
import schema from "../schema.js";
const model = mongoose.model("CourseModel", schema);
export default model;
```

Similarly the **Users model** below declares the name of the model as **UserModel** for **user** documents stored in the **users** collection.

```
import mongoose from "mongoose";           // Load mongoose library
import schema from "../schema.js";          // Load users schema
const model = mongoose.model("UserModel", schema); // create mongoose model from the schema
export default model;                       // export so it can be used elsewhere
```

In a new **schema** file shown below, implement a **many to many Enrollments** relationship that relates **user** and **course** documents stored in the **users** and **courses** collections, referred to by their model names **CourseModel** and **UserModel** respectively.

```
import mongoose from "mongoose";
const enrollmentSchema = new mongoose.Schema(
  {
    _id: String,
    course: { type: String, ref: "CourseModel" },
    user: { type: String, ref: "UserModel" },
    grade: Number,
    letterGrade: String,
    enrollmentDate: Date,
    status: {
      type: String,
      enum: ["ENROLLED", "DROPPED", "COMPLETED"],
      default: "ENROLLED",
    },
  },
  { collection: "enrollments" }
);
export default enrollmentSchema;
```

Create an **Enrollments model** file as shown below, to **CRUD** enrollment documents in an **enrollments** collection.

```
import mongoose from "mongoose";
import schema from "../schema.js";
const model = mongoose.model("EnrollmentModel", schema);
export default model;
```

Create an **Enrollments DAO** file that implements operations that create enrollments, deletes enrollments, and filters enrollments by either a course or user. The following sections describe each of the operations in detail.

```
import model from "../model.js";
export default function EnrollmentsDao(db) {
  async function findCoursesForUser(userId) {
    const enrollments = await model.find({ user: userId }).populate("course");
    return enrollments.map((enrollment) => enrollment.course);
  }
  async function findUsersForCourse(courseId) {
    const enrollments = await model.find({ course: courseId }).populate("user");
    return enrollments.map((enrollment) => enrollment.user);
  }

  function enrollUserInCourse(userId, courseId) {
    return model.create({
      user: userId,
      course: courseId,
      _id: `${userId}-${courseId}`,
    });
  }
  // const [enrollments] = db;
  // enrollments.push({
  //   _id: uuidv4(),
```

```

// user: userId,
// course: courseId,
// });
}
function unenrollUserFromCourse(user, course) {
  return model.deleteOne({ user, course });
}

return {
  findCoursesForUser,
  findUsersForCourse,
  enrollUserInCourse,
  unenrollUserFromCourse,
};
}

```

6.4.3.2 Retrieving Courses for Enrolled Users

Enrollments establish a *many to many relationship* between **users** and **courses**. A common operation consists of finding which documents in one collection are related to documents in the other collection. For instance, given a particular user we would like to determine which courses are related to that user, e.g., which courses is a user enrolled in. The **findCoursesForUser()** function below retrieves the **enrollment** documents for a given **user**. The **enrollments** documents would contain the **primary keys** for the **user** and **course** documents being referenced. The **populate()** function tells **mongoose** to use the value of the **primary keys** to fetch the actual document referenced by the **key**. The **populate("course")** function replaces the **course primary key** value in the **enrollments** document with the actual **course** document from the **courses** collection corresponding to the **key's** value. The **enrollments.map()** operation unwraps the **enrollments** array and returns a new array with just the **course** objects.

kambaz/enrollments/dao.js

5

```

import model from "../model.js";
export async function findCoursesForUser(userId) {
  const enrollments = await model.find({ user: userId }).populate("course");
  return enrollments.map((enrollment) => enrollment.course);
}
...

```

In the **Courses routes**, refactor route function **findCoursesForEnrolledUser** to retrieves courses for a given user using the new **findCoursesForUser** in the **Enrollments DAO** as shown below. Login and confirm that the **Dashboard** displays the courses for the logged in user. Also confirm that creating new courses inserts new enrollments in the database.

kambaz/courses/routes.js

5

```

const findCoursesForEnrolledUser = async (req, res) => {
  let { userId } = req.params;
  if (userId === "current") {
    const currentUser = req.session["currentUser"];
    if (!currentUser) {
      res.sendStatus(401);
      return;
    }
    userId = currentUser._id;
  }
  // const courses = await dao.findCoursesForEnrolledUser(userId);
  const courses = await enrollmentsDao.findCoursesForUser(userId);
  res.json(courses);
};

```

6.4.3.3 Deleting Courses

In the **Enrollments DAO**, implement a new **unenrollAllUsersFromCourse** function that removes all enrollments for a given course.

kambaz/enrollments/dao.js

S

```
function unenrollAllUsersFromCourse(courseId) {  
  return model.deleteMany({ course: courseId });  
}  
  
return {  
  findCoursesForUser,  
  findUsersForCourse,  
  enrollUserInCourse,  
  unenrollUserFromCourse,  
  unenrollAllUsersFromCourse,  
};
```

In the **Courses DAO**, remove all uses of the **enrollments** from the **db** since we are just going to interact with enrollments in **MongoDB**.

kambaz/courses/dao.js

S

```
function deleteCourse(courseId) {  
  // const { courses, enrollments } = db;  
  // db.courses = courses.filter((course) => course._id !== courseId);  
  // db.enrollments = enrollments.filter(  
  //   (enrollment) => enrollment.course !== courseId  
  // );  
  return model.deleteOne({ _id: courseId });  
}
```

In the **Courses routes**, unenroll all users when the course is deleted. Signin and try removing a course. Confirm that the course is removed from the database and that all enrollments are removed for the course.

kambaz/courses/routes.js

S

```
const deleteCourse = async (req, res) => {  
  const { courseId } = req.params;  
  await enrollmentsDao.unenrollAllUsersFromCourse(courseId);  
  const status = await dao.deleteCourse(courseId);  
  res.send(status);  
};
```

6.4.3.4 Enrolling / Unenrolling (On Your Own)

In a prior assignment you implemented enrolling and unenrolling users from courses. The implementation relied on manipulating arrays of courses, users, and enrollments in a "Database" data structure. Now that we moved the data to an actual database, refactor your implementation to use enrollments stored in the database. The **Enrollments DAO** implements **enrollUserInCourse** and **unenrollUserFromCourse** functions as shown below. The **enrollUserInCourse** function inserts a new **enrollment** document in the **enrollments** collection creating a relation between a user and the course they are enrolled in. The **unenrollUserFromCourse** function deletes an existing **enrollment** document from the **enrollments** collection, removing the relation between a user and the course they were enrolled in.

kambaz/enrollments/dao.js

S

```
function enrollUserInCourse(userId, courseId) {  
  return model.create({  
    user: userId,
```

```

    course: courseId,
    _id: `${userId}-${courseId}`,
  });
}
function unenrollUserFromCourse(user, course) {
  return model.deleteOne({ user, course });
}

```

In the **Courses routes** implement **post** and **delete** routes that create or remove an enrollment using the corresponding **DAO** functions. If you had already implemented these functions in a prior assignment, feel free to use those functions instead or refactor at your own discretion.

kambaz/courses/routes.js

5

```

import CourseDao from "../dao.js";
import EnrollmentsDao from "../enrollments/dao.js";
export default function CourseRoutes(app, db) {
  const dao = CourseDao(db);
  const enrollmentsDao = EnrollmentsDao(db);
  ...
  const enrollUserInCourse = async (req, res) => {
    let { uid, cid } = req.params;
    if (uid === "current") {
      const currentUser = req.session["currentUser"];
      uid = currentUser._id;
    }
    const status = await enrollmentsDao.enrollUserInCourse(uid, cid);
    res.send(status);
  };
  const unenrollUserFromCourse = async (req, res) => {
    let { uid, cid } = req.params;
    if (uid === "current") {
      const currentUser = req.session["currentUser"];
      uid = currentUser._id;
    }
    const status = await enrollmentsDao.unenrollUserFromCourse(uid, cid);
    res.send(status);
  };
  app.post("/api/users/:uid/courses/:cid", enrollUserInCourse);
  app.delete("/api/users/:uid/courses/:cid", unenrollUserFromCourse);
  app.get("/api/users/:userId/courses", findCoursesForEnrolledUser);
  ...
}

```

In the React application, create client functions **enrollIntoCourse** and **unenrollFromCourse** that request the server to enroll or unenroll a user from a course. Encode the primary keys of the **user** and **course** as part of the path. If you already had implemented these functions in a prior assignment, feel free to use those functions instead or refactor at your discretion.

app/(kambaz)/courses/client.ts

```

import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
export const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export const USERS_API = `${HTTP_SERVER}/api/users`;
...
export const enrollIntoCourse = async (userId: string, courseId: string) => {
  const response = await axiosWithCredentials.post(`${USERS_API}/${userId}/courses/${courseId}`);
  return response.data;
};
export const unenrollFromCourse = async (userId: string, courseId: string) => {
  const response = await axiosWithCredentials.delete(`${USERS_API}/${userId}/courses/${courseId}`);
  return response.data;
};

```

On your own, refactor the enrollment/unenroll logic implemented in earlier assignments in the React user interface. The new implementation should use the new client functions to allow students to enroll and unenroll from a course. Confirm users can enroll and unenroll from courses and that the corresponding enrollment documents are inserted and removed from the database.

6.4.3.5 Retrieving Students Enrolled in a Course (On Your Own)

The **Users** screen implemented in an earlier assignment, uses the **PeopleTable** to display all the users in the database and allows admin users to create, read, update, and delete all users in the database. The **People** link in the **Courses Navigation** in the **Courses** page, should display a page that lists all the users enrolled in the current course. Reimplement the **People** link so that the **PeopleTable** only displays the users that are enrolled in a particular course when navigating to the **People** link. In the **Courses routes** use the **enrollments DAO findUsersForCourse** function to retrieve the users enrolled in a course.

kambaz/courses/routes.ts

5

```
import * as enrollmentsDao from "../enrollments/dao.js";
export default function CourseRoutes(app) {
  ...
  const findUsersForCourse = async (req, res) => {
    const { cid } = req.params;
    const users = await enrollmentsDao.findUsersForCourse(cid);
    res.json(users);
  }
  app.get("/api/courses/:cid/users", findUsersForCourse);
  ...
}
```

In **Courses client** implement **findUsersForCourse()** client function to retrieve the users for a given course. Use the **Courses routes** and **client** to display the users enrolled in a course when navigating to a course's **People** route.

app/(kambaz)/courses/client.ts

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
const COURSES_API = `${HTTP_SERVER}/api/courses`;

export const findUsersForCourse = async (courseId: string) => {
  const response = await axios.get(`${COURSES_API}/${courseId}/users`);
  return response.data;
};
```

6.4.4 Assignments (On Your Own)

Implement **schema**, **model**, **DAO**, **routes**, and **client** files so that the **Assignments** and **AssignmentsEditor** screens display assignments stored in a database. Users should be able to display assignments in a course, create new assignments, update assignments, and delete existing assignments. Confirm that all operations are reflected in the database.

6.5 Deliverables

As a deliverable, make sure you complete all the lab exercises, mongoose schemas, models, DAOs, React components, and they behave as described. For both the React and Node repositories, all your work should be done in a branch called **a6**. When done, add, commit and push both branches to their respective GitHub repositories. Deploy the new branches to **Vercel** and **Render** (or **Heroku**) and confirm they integrate. If you are using **Render**, it does not create a separate branch deployment like **Vercel** so you'll have to deploy to an entirely different Web service so that you don't trample the previous assignment while the TAs are still grading. The Node server application running on Render will need to be configured to interact with the **a6 Vercel** branch deployment. All the exercises should work remotely just as well as locally. The Kambaz Copyright © 2026 Jose Annunziato. All rights reserved.

Dashboard should display the courses and modules from the database. As a deliverable in **Canvas**, submit the URL to the **a6** branch deployment of your React application running on Vercel.